OUTAGE DISSECTIONS • DATABASE FAILOVER • GTID DIVERGENCE

Dissecting GitHub Outage: When Automated Master Failover Fails

Curated from Arpit Bhayani's Architecture Dissection Series • Core Focus: Errant Transactions & Failover Deadlocks

EXECUTIVE ARCHITECTURAL ABSTRACT

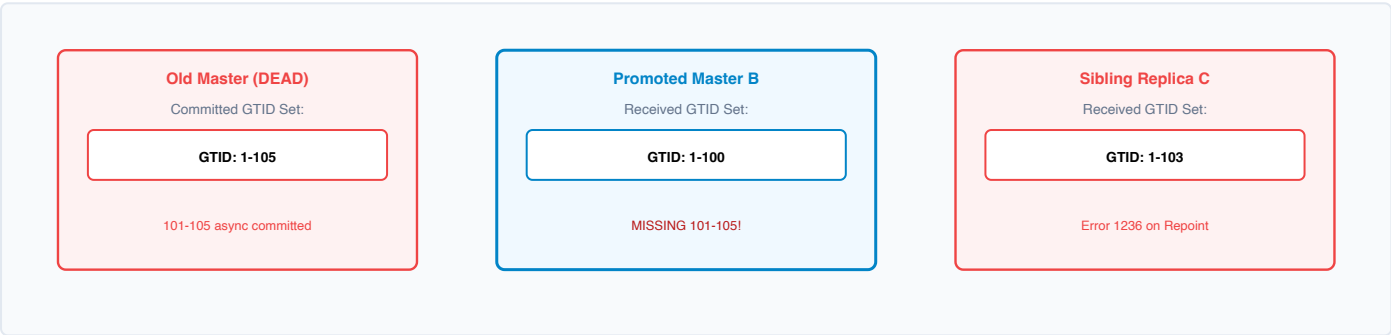
Automated database failovers are intended to maintain high availability by seamlessly promoting a replica when a primary master fails. However, when failover automation executes while replication is silently degraded, it can trigger an **Errant Transaction Deadlock**. In this GitHub incident, semi-synchronous replication timed out, causing the master to fall back to asynchronous mode. Upon master crash, a lagging replica was promoted. Surviving replicas refused to replicate from the new master due to missing GTID sequences, causing replication to fail completely across the entire cluster.

1. THE ANATOMY OF AN AUTOMATED FAILOVER DEADLOCK

Automated failover systems (like GitHub Orchestrator) assume that a promoted replica possesses a strict superset of all transactions committed in the cluster. When this invariant is broken, severe failure states arise:

- **Semi-Sync Fallback:** If semi-sync replication times out, the master commits writes locally without replica receipt.
- **Incomplete Master Promotion:** Orchestrator promotes a replica that is missing the latest asynchronous transactions.
- **GTID Rejection:** Other surviving replicas that received those transactions from the old master reject the new master because it lacks their history (Error 1236).

2. VISUALIZING ERRANT TRANSACTION DIVERGENCE



3. THE MECHANICS OF THE GTID REPOINTING ERROR

MySQL Global Transaction Identifiers guarantee cluster consistency by enforcing mathematical set ordering:

1. The Divergent Set Comparison

When Replica C connects to Promoted Master B, MySQL calculates the missing transaction set: `Diff = Replica_C_GTIDs - Master_B_GTIDs`. Because Replica C contains transactions 101, 102, 103, which Master B never witnessed, `Diff` is non-empty.

2. Immediate Replication Abort

MySQL refuses to replicate from a master that lacks a slave's committed history. Slave I/O thread crashes with `Fatal Error 1236`. All sibling replicas abort replication, leaving Master B completely isolated.

4. SURGICAL REMEDIATION: EMPTY GTID INJECTION

To break the replication deadlock without rebuilding terabyte-scale replicas from scratch, engineers perform **GTID Injection**:

Placeholder Injection SQL Script

```
-- 1. Identify missing transactions on Promoted Master
-- 2. Inject empty placeholder transactions to satisfy GTID set requirement
SET GTID_NEXT = '3E11FA47-71CA-11E1-9E33-C80AA9429562:101';
BEGIN; COMMIT;
SET GTID_NEXT = '3E11FA47-71CA-11E1-9E33-C80AA9429562:102';
BEGIN; COMMIT;
SET GTID_NEXT = '3E11FA47-71CA-11E1-9E33-C80AA9429562:103';
BEGIN; COMMIT;
SET GTID_NEXT = 'AUTOMATIC';

-- 3. Restart slave thread on surviving replicas
START SLAVE;
```

By committing dummy transactions on Master B matching the missing IDs, Master B's GTID set is artificially inflated to include 1–103. Sibling replicas accept Master B, and replication resumes.

5. HARDENING FAILOVER CONFIGURATIONS

Preventing errant transaction deadlocks requires tightening MySQL replication variables:

- **Zero Semi-Sync Timeout:** Set `rpl_semi_sync_master_timeout = 1000000000`. Writes must block if replicas disconnect, rather than silently falling back to asynchronous mode.
- **Strict Candidate Prioritization:** Configure Orchestrator to reject candidates that do not possess the strict maximal GTID set across all alive replicas.
- **Automated Pre-Failover Fencing:** Before promoting any replica, enforce network-level fencing of the old master to ensure no trailing transactions slip into binlogs.

6. ARCHITECTURE MATRIX: FAILOVER PROTOCOLS UNDER NETWORK DROPS

Replication Strategy	Network Blip Behavior	Failover Risk	Data Loss Probability
Asynchronous Replication	Master continues committing writes	High (Promotion loses un-replicated binlogs)	High (RPO > 0)
Semi-Sync with Fallback	Falls back to async after 1000ms timeout	**Extreme (Deadlocks via errant GTIDs)**	Moderate (Silent vulnerability window)
Strict Semi-Sync (No Fallback)	Master stalls writes until replica confirms	**Zero (All replicas stay in sync)**	**Zero (RPO = 0)**
Consensus Quorum (Raft / Paxos)	Requires majority quorum for commits	**Zero (Mathematical consistency by design)**	**Zero (RPO = 0)**

7. PRODUCTION ENGINEERING TAKEAWAYS

1. **Beware of Silent Degradation Modes:** Default MySQL semi-sync settings prioritize availability over consistency by falling back to async. In production financial or mission-critical systems, silent fallbacks cause unrecoverable split-brain.
2. **Automate Candidate GTID Auditing:** An orchestrator must audit the entire cluster before promoting. If Node B has GTIDs 1–100 and Node C has GTIDs 1–103, promoting Node B is a fatal mistake. Always promote Node C or inject transactions into Node B first.
3. **Consensus Storage Eliminates Errant Transactions:** Modern databases (CockroachDB, Spanner, TiDB) avoid errant transactions entirely because log replication is governed by Raft/Paxos quorums rather than point-to-point binlog streaming.